

JAVACREAM

*Training
Consulting
Projectmanagement*

Python



```
-----  
tn015.raum01@cegos.de  
tn016.raum01@cegos.de  
tn017.raum01@cegos.de  
tn018.raum01@cegos.de  
tn019.raum01@cegos.de  
tn020.raum01@cegos.de  
tn021.raum01@cegos.de  
tn022.raum01@cegos.de  
tn023.raum01@cegos.de  
tn024.raum01@cegos.de  
tn025.raum01@cegos.de  
tn026.raum01@cegos.de  
tn027.raum01@cegos.de  
tn028.raum01@cegos.de  
tn029.raum01@cegos.de  
tn030.raum01@cegos.de
```

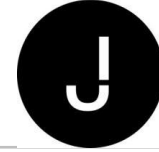
Bautista, Osorio
Campitelli, Natalia
Dieling, Johannes
Eichenlaub, David
Franke, Elena
Gudehus, Jan
Heinrich, Susanne
Huppmann, Ulrich
Kamm-Hartung, Barbara
Meyer, Gregor
Nedeltcheva, Lubomira
Pournaghi, Seyedali
Rother, Judith
Vogel, Gesa
Witzlsperger, Andreas

- Name
- Role in the company
- Prior knowledge of the topic
- Current problem
- Concrete individual objective

- Python Installation
 - Eine Ausführungsschicht
 - Check: In einem Terminal erfolgt der Aufruf des Python Interpreters
 - `python -V`
 - `python3 -V`
- Entwicklungsumgebung
 - Unterstützung zur reinen Programmierung
 - Editor mit Syntax Highlighting, Code Assist
 - Debugger
 - Visual Studio **Code**
- Freier Zugriff auf Internet-Ressourcen

Overview and first contact

- Python is a "completely normal" programming language
 - Variables, operators, control structures, functions, ...
 - Syntactic peculiarities are already present...
 - Indentations are syntactically important
 - "Syntactic Sugar" = elegant programming
 - Comprehension
 - Lambda function
- Python is currently in widespread use
 - Number 1, better: Top 3
 - This gives us a large community with resources such as tutorials, documentation, best practices

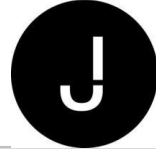


- install Python extension from Microsoft
- first.py

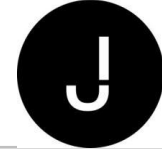
- read data
- calculate result
- write result

```
height = 183
weight = 75.7
body_mass_index = weight / (height*height) * 10000
print(body_mass_index)
```

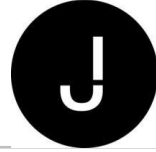
```
data = pd.read_sql("sql statement")
data.fillna("value")
data.hist()
```

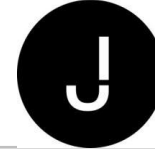
- variables
 - name
 - convention
 - only lower case
 - underscores
 - english
- value assignment
 - = value
- value types
 - height = 183 # 183 is an integer number
 - weight = 76.6 # 76.6 is a float number
- calculations using math operators
 - +, -, *, /
 - brackets
- use the print function to print a variable to terminal



- strings
 - `name = "Sawitzki" # "..."`
 - in Python more frequent: `name = 'Sawitzki'`
- formatted string
 - `f'...'`
 - `{ }`

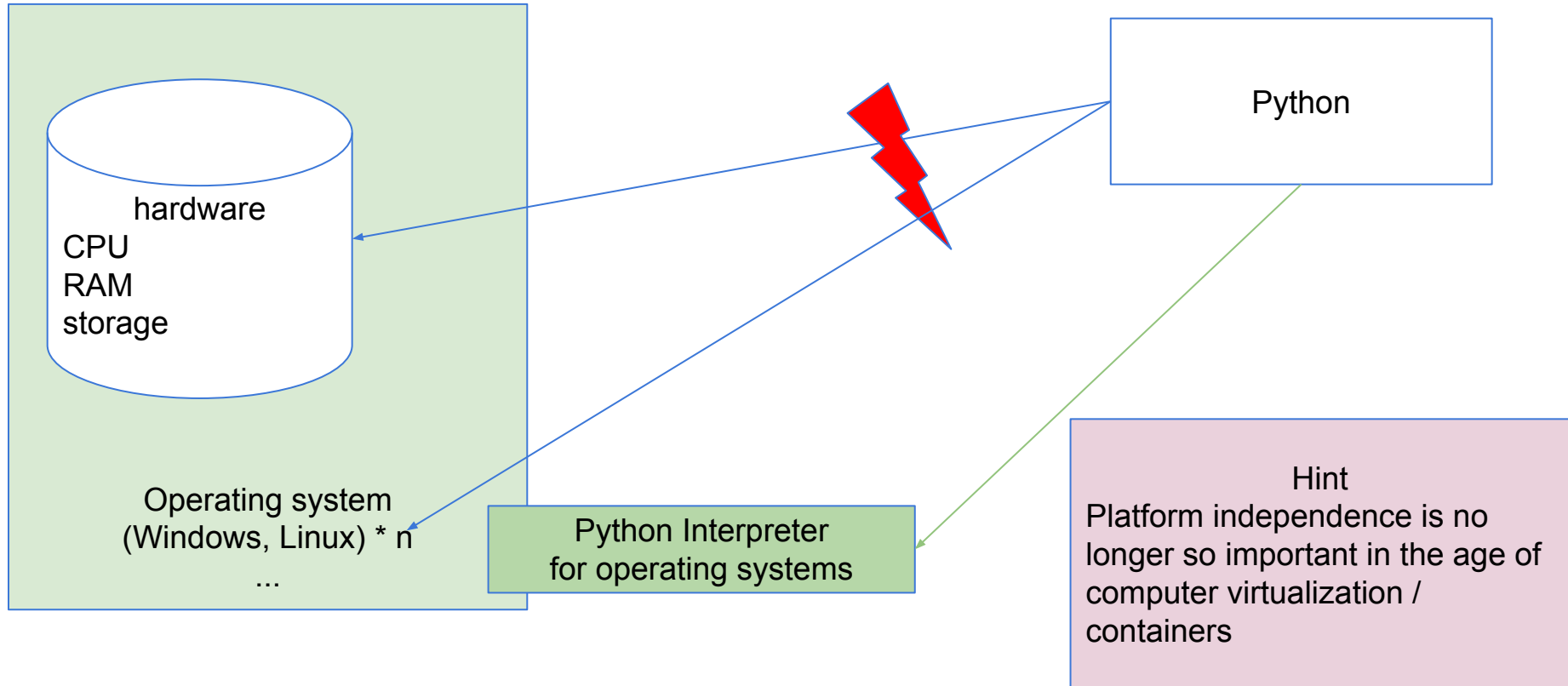


- user input
 - `input(prompt)`
- caution:
 - input returns always a string
 - you cannot do math calculations with strings!
- type conversion
 - `int(value)`
 - `float(value)`

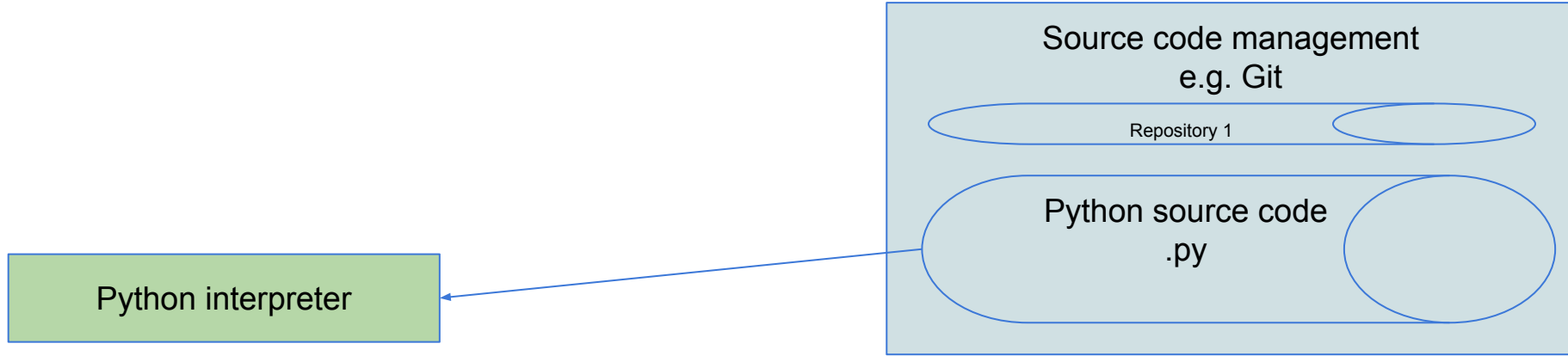


Python

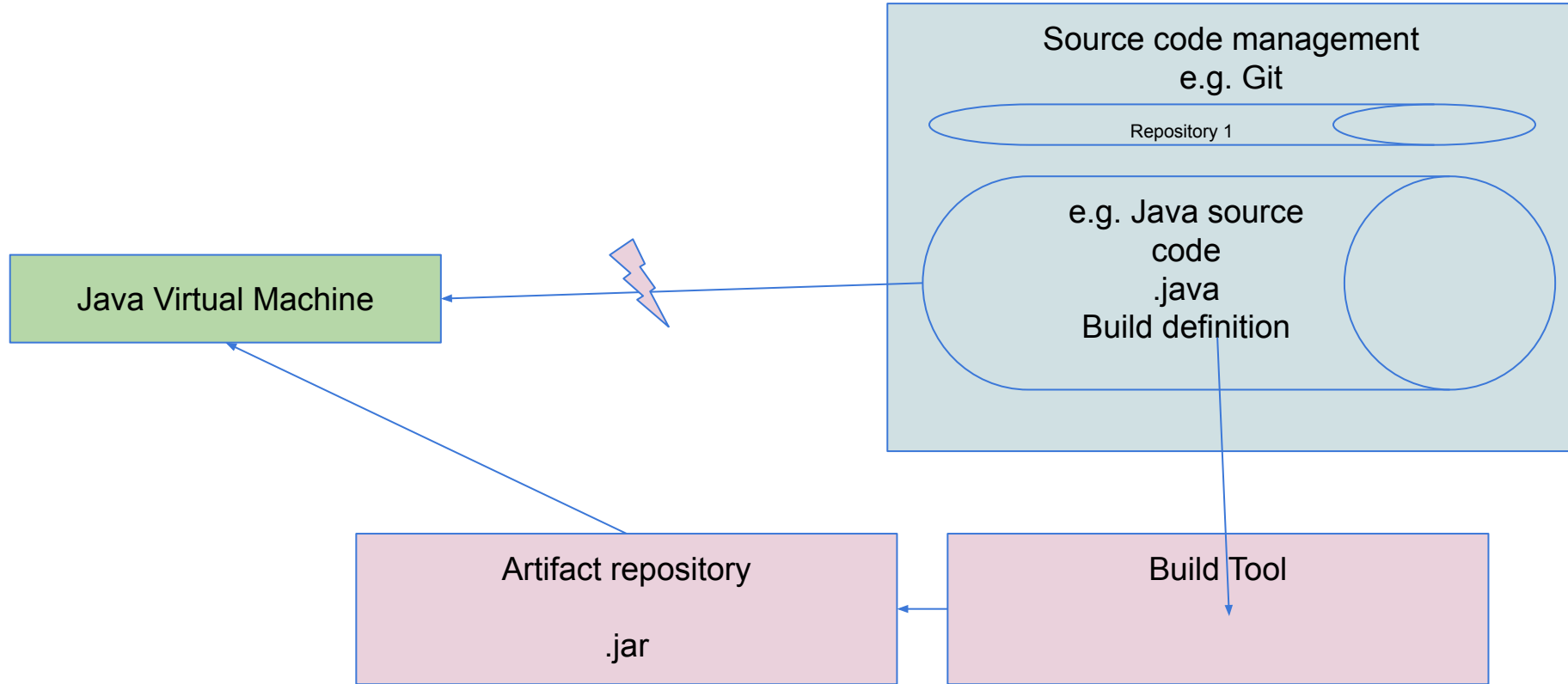
Platform independence

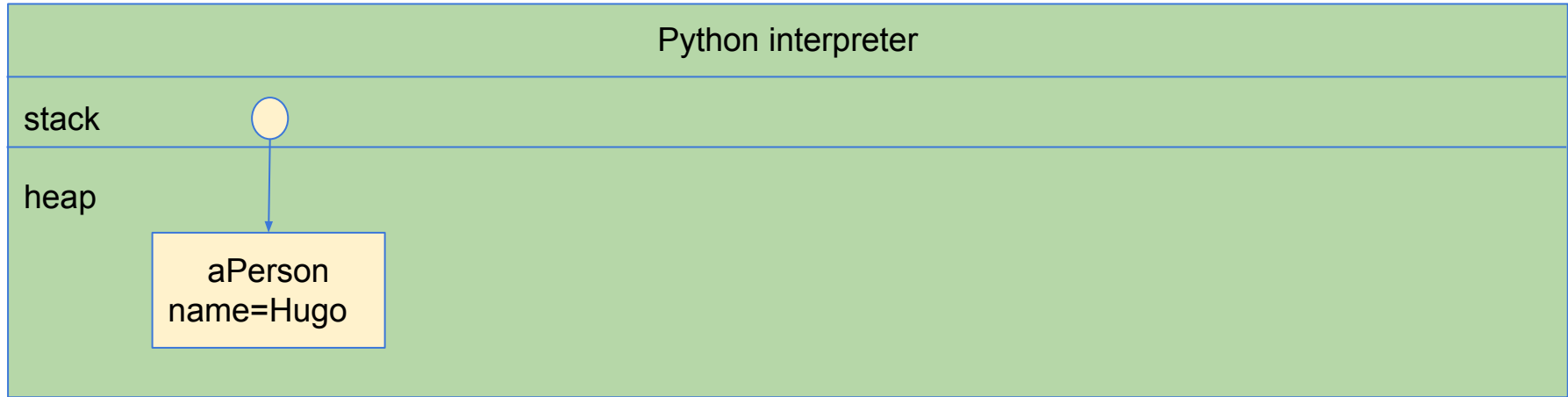


A script language

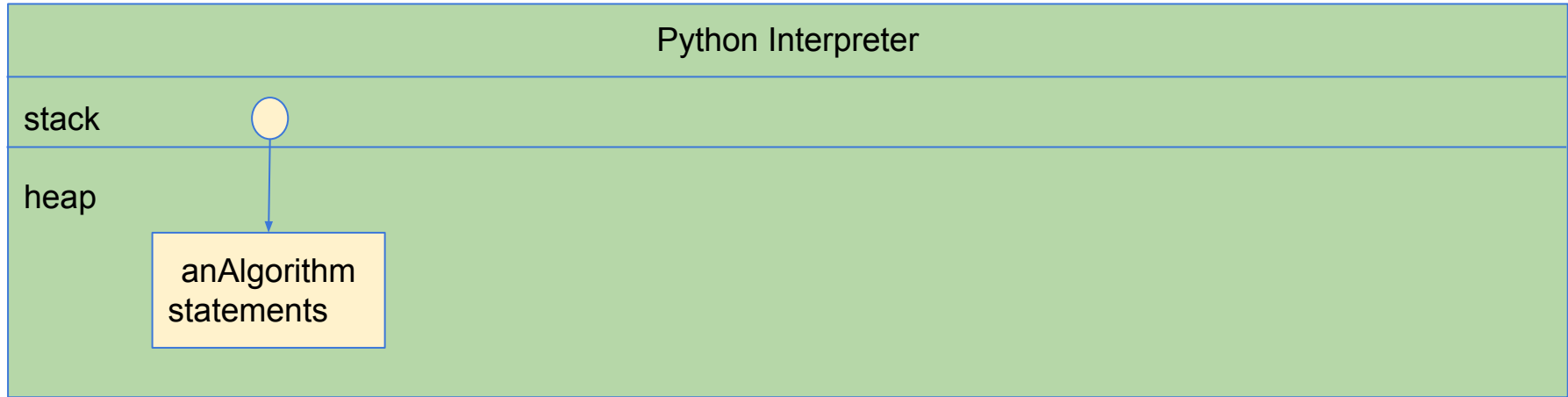


Excursus: Compiler language





- + Stack is the domain of programming
 - + variables
- + Objects are created and managed internally in the heap (especially garbage collection)
- + Variables are references to objects

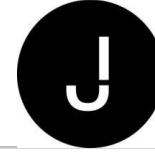


+ Algorithms = functions are also objects

- Untyped
 - `data = 42`
 - data is therefore the number with the value 42
 - `data = 'Hugo'`
 - data is then a character string with the value 'Hugo'
- Open means that the structure / data schema of an object can be changed at runtime

Notes

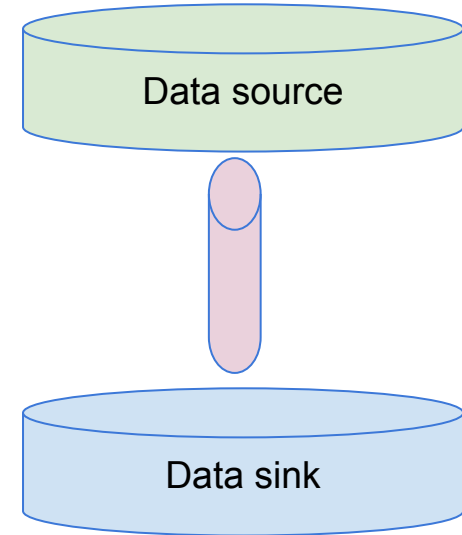
- + Both untyped and open are critically discussed in the developer community
- + AddOn "typing" introduces so-called type hints, `data:int = 42`
- + Objects can be "frozen" programmatically and are therefore statically fixed



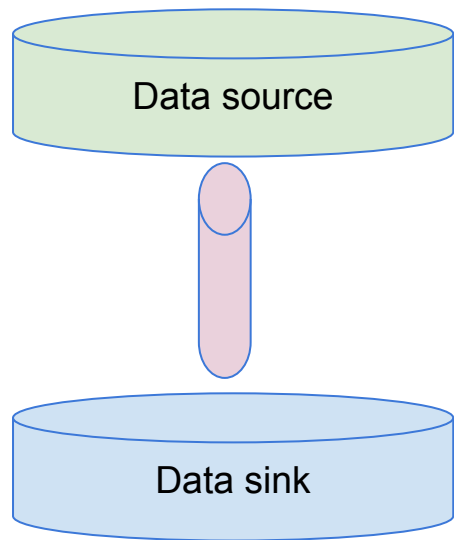
First Contact

- Visual Studio Code with Python Extension
 - an integrated development environment
 - file explorer
 - Terminal window
 - editor
 - Syntax highlighting
 - Automatic check (~ spell check)
 - Code Assist with suggestions "what makes sense next?"
 - Integration of AI (Copilot, ChatGPT, ...)
 - **However, far too extensive at the current stage of learning Python**
 - Convenient start of the program

- Read in data
- Data processing
 - Checking / validation
 - Cleansing
 - Filtering
 - Transforming
 - Calculating / Aggregating
- Writing the results



- Reading data **through variables**
- Data processing
 - Check / Validation
 - Cleansing
 - Filtering
 - Transforming
 - Calculating **with operators / aggregating**
- Writing the results **to the console**



- Syntax error
 - In VSC with a relatively high probability of under-ringing
 - The program does not run at all, an error is output immediately
- Runtime error
 - VSC may be able to recognize such errors
 - NameError, often a mistyping of variable names is recognized well
 - Instead of 74.1 74,1 (wrong separator) -> ValueError with cryptic error message and hint takes the wrong line number
- Logical errors
 - e.g. an error in the calculation formula
 - Only testing the application and a code review will help here

- Programming with Copilot / ChatGPT
 - Caution: This presentation may cause you to leave the course because "Chat can already do everything"

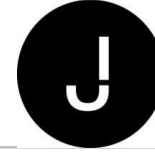
- Visual Studio Code with Python Extension
 - an integrated development environment
 - file explorer
 - terminal window
 - editor
 - Syntax highlighting
 - Automatic check (~ spell check)
 - Code Assist with suggestions "what makes sense next?"
 - The integration of AI (Copilot, ChatGPT, ...) is available for a fee
However, far too extensive at the stage of learning Python
 - Convenient start of the program
 - Debugger
 - Stops at the breakpoints
 - and visualizes all current variables

- Values of a program are defined by "literals"
 - 'Meier', "Meier" -> character strings
 - 183 -> integer
 - 74.1 -> Floating point number
- Hint
 - There are a few more literals -> later
- Type information of a variable is obtained with the type function
 - `print(type(name))` -> `<class 'str'>`

Hint

- + `print` and `type` are so-called built-in functions in Python that are always available
- + There are around 60 in total, of which around 10 are used in the seminar

- Python documentation
 - <https://docs.python.org/3/>
 - Very technical, unusable in the current state of the seminar
- Tutorial
 - <https://www.w3schools.com/python/>
 -



Control structures

- Control structures for conditional program execution
 - if - elif - else
 - ~~switch-match-case~~ -> later, or documentation
- Loops
 - ~~for (index = 0; index < 42; index = index + 1)~~
 - while
 - ~~do while~~
 - for element in container
 - later, we need data containers for this
- Program execution with error handling
 - try - except
 - Detailed only after OOP introduction, but pragmatically "soon"

- bmi categorization

```
if body_mass_index < 18
```

```
    bmi_category = 'underweight'
```

```
if body_mass_index >= 18 and body_mass_index < 25
```

```
    bmi_category = 'normalweight'
```

```
if body_mass_index >= 25 and body_mass_index < 30
```

```
    bmi_category = 'overweight'
```

```
if body_mass_index >= 30
```

```
    bmi_category = 'obese'
```

- Not (!) block-oriented
 - if number < 9{
 - Instructions for number is less than 9
 - }
 - else {
 - Instructions for number greater than or equal to 9
 - }

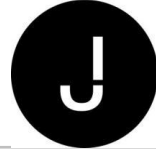
- Not (!) block-oriented
 - if number < 9
 - {
 - Instructions for number is less than 9
 - }
 - else
 - {
 - statements for number greater than or equal to 9
 - }

- Not (!) block-oriented
 - if number < 9{instructions for number is less than 9}
 - else
 - {instructions for number greater than or equal to
 - }

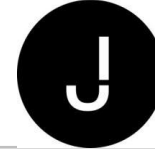
Still correct, but brutally hard to read -> "Format Code"

- if number < 9:
 - Instructions for number is less than 9
- else:
 - Instructions for number greater than or equal to 9
-

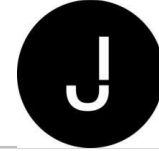
Python does not work like this, but takes the indentations, which are necessary for readability anyway, as a block criterion



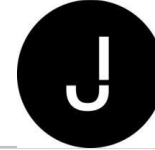
- have a name
 - like variables
- have an implementation
 - sequence of instructions
- the implementation is a transformation of parameters into a result
 - the result is the return value of a function
 - syntax: function parameters are included in round brackets



Collections



- list
 - container for elements
 - access using numerical index
 - [element1, element2]
 - iteration with for element in collection



- programming means:
 - we are calling "actions"
 - functions
 - calling a function is done using round brackets
 - list
 - action "give me element at index"
 - this action is called using square brackets

- objects are used all the time in our daily environment
- every objects has
 - attributes
 - methods
 - a method is a function, that works in the object context
 - built-in functions in python (print, input, int, float, ...) are usable "everywhere"
 - methods group algorithms that are useful for a type
 - example
 - append an element to a list

object oriented syntax
object.attribute
object.method()

In Python is everything an object
therefore every variable is an object

- Online documentation
 - official site
 - <https://docs.python.org/3/tutorial/datastructures.html>
 - better: e.g. w3school
 - https://www.w3schools.com/python/python_lists_methods.asp

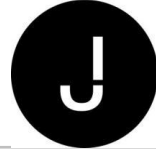


- a string is a type
 - `content = 'Hello'`
 - til now: content is a variable of type `str`
 - now: content is an object of type `str`
 - consequence: every string has methods
- a string is a list of characters
 - accessing these characters uses an index
 - `content[2]`

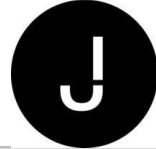
- how many characters has the string "Hello my friend"
- how many characters 'e' has ...
- how many words has the sentence "Hello my friend"
 - Hint: words are separated by whitespace
- is the string "123" numeric?
 - is it possible to convert it to an int
 - optional: this check can be used in [bmi.py](#)
- is the string "12.3" numeric?
 - is it possible to convert as float
- ...

Hint

if you find the right method
the solution is simple



- not mandatory for Python programming
- help for VSC code assist



- range
 - from - to - step
 - used for iterations
- set
 - contains elements without duplicates
 - math operations like union, intersection
- dictionary
 - lookup table

- we know
 - `names[2]` results in the third element in `names`
- new: slicing
 - `names[1:3]` results in a sublist with elements with index 1, 2

- expectation: one + x-liner
 - `file = open('path')` # better: `open('path', 'rt', encoding='utf-8')`
 - `content = file.read()`
 - `file.close()`
- todo
 - read file sample
 - `read()` vs. `readline()` vs `readlines()`
 - `'\n'` = new_line
 - revisited: slide 42
 - analysis of a text files
 - number of lines, number of words, ...
 - number of unique words?
 - case sensitive or case insensitive

unusal, better alternative ->
`file_scratch.py`